



NUST CHIP DESIGN CENTRE

RISC-V Assembly

Lab Manual # 12

Introduction to RISC-V Assembly & Custom Processor Design

<u>Name</u>	<i><u>Muddassir Ali Siddiqui</u></i>
<u>Instructor</u>	<i><u>Sir Imran & Sir Umer</u></i>
<u>Date</u>	<i><u>13th Aug 2025</u></i>

1. In-Lab Tasks: (Write your lab task & screenshots here)

Part 1:

i. Task1

```
churan@churan-ThinkPad-X270-W10DG:~/Downloads/muddassir$ ls
hello.c
churan@churan-ThinkPad-X270-W10DG:~/Downloads/muddassir$ gcc ./hello.c -o ./hello.x86.asm -S
churan@churan-ThinkPad-X270-W10DG:~/Downloads/muddassir$ ls
hello.c  hello.x86.asm
churan@churan-ThinkPad-X270-W10DG:~/Downloads/muddassir$ riscv64-unknown-elf-gcc hello.c -o hello.rv64.bin
churan@churan-ThinkPad-X270-W10DG:~/Downloads/muddassir$ ls
hello.c  hello.rv64.bin  hello.x86.asm
churan@churan-ThinkPad-X270-W10DG:~/Downloads/muddassir$ riscv64-unknown-elf-gcc hello.c -o hello
churan@churan-ThinkPad-X270-W10DG:~/Downloads/muddassir$ ls
hello  hello.c  hello.rv64.bin  hello.x86.asm
churan@churan-ThinkPad-X270-W10DG:~/Downloads/muddassir$ riscv64-unknown-elf-gcc -o hello.rv64.elf hello.c
churan@churan-ThinkPad-X270-W10DG:~/Downloads/muddassir$ ls
hello  hello.c  hello.rv64.bin  hello.rv64.elf  hello.x86.asm
churan@churan-ThinkPad-X270-W10DG:~/Downloads/muddassir$ riscv64-unknown-elf-objdump -s hello.rv64.elf > hello.rv64.hex
churan@churan-ThinkPad-X270-W10DG:~/Downloads/muddassir$ ls
hello  hello.c  hello.rv64.bin  hello.rv64.elf  hello.rv64.hex  hello.x86.asm
churan@churan-ThinkPad-X270-W10DG:~/Downloads/muddassir$ riscv64-unknown-elf-gcc -S hello.c -o hello.s
churan@churan-ThinkPad-X270-W10DG:~/Downloads/muddassir$ ls
hello  hello.c  hello.rv64.bin  hello.rv64.elf  hello.rv64.hex  hello.s  hello.x86.asm
churan@churan-ThinkPad-X270-W10DG:~/Downloads/muddassir$ S
```

ii. Task 2

1. Unique Assembler Directives

Directive	Purpose
.file "hello.c"	Records the source file name for debugging.
.option nopic	Disables Position Independent Code (absolute addresses instead).
.attribute arch, ...	Specifies target RISC-V architecture and extensions.
.attribute unaligned_access, 0	Declares unaligned memory access is not supported.
.attribute stack_align, 16	Ensures stack is always 16-byte aligned (ABI requirement).
.text	Marks start of the text (code) section .
.section .rodata	Switches to read-only data section for constants/strings.
.align 3	Aligns data to an 8-byte boundary ($2^3 = 8$).
.string "Muddassir_Ali_Siddiqui"	Stores a null-terminated string constant.
.align 2	Aligns following code/data to 4-byte boundary ($2^2 = 4$).
.globl main	Makes main visible to linker as a global symbol.
.type main, @function	Declares main as a function symbol.
.size main, .-main	Defines the size of main function.
.ident "GCC: ..."	Embeds compiler identification string.
.section .note.GNU-stack,"",@progbits	Marks stack as non-executable for security.

2. Unique Base Instructions

addi

lui

ld

sd

3. Unique Labels

.LCO:

main:

4. Unique Pseudo instruction and their corresponding base instructions.

call printf	auipc ra, %pcrel_hi(printf); jalr ra, %pcrel_lo(printf)(ra)
li a5, 0	addi a5, x0, 0
mv a0, a5	addi a0, a5, 0
jr ra	jalr x0, 0(ra)

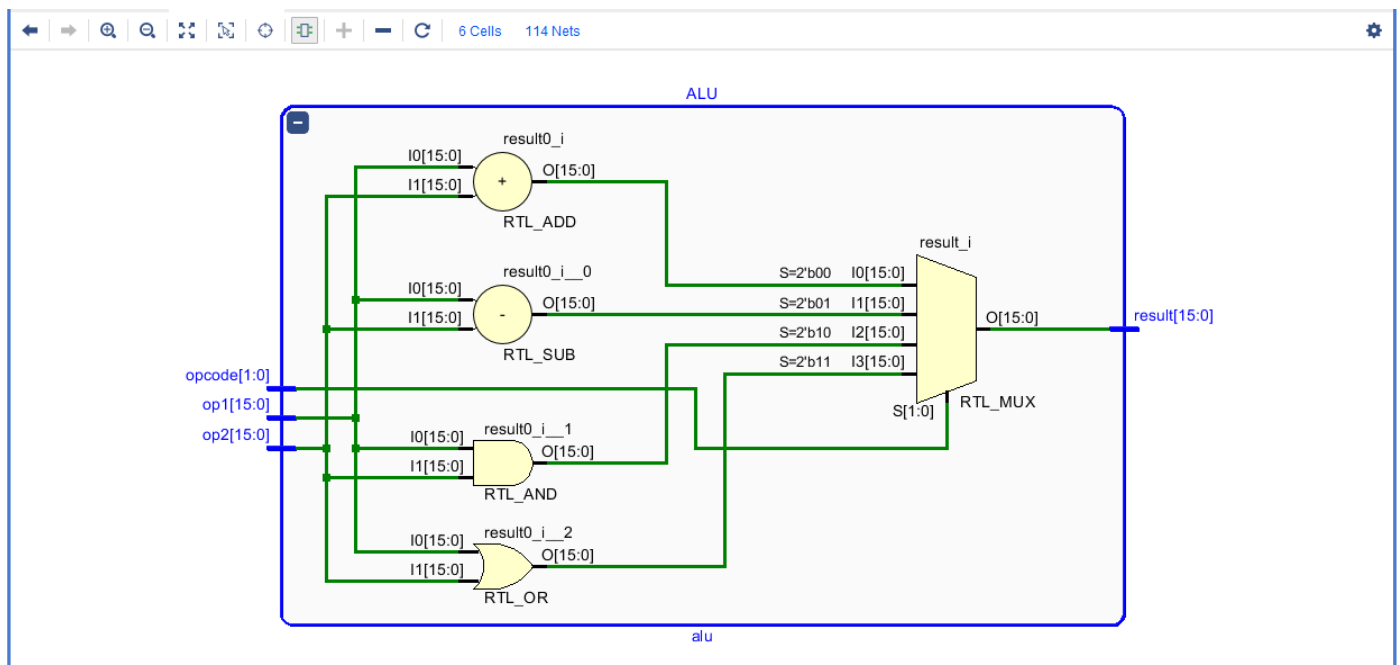
iii. Task 3

Contents of section .rodata:

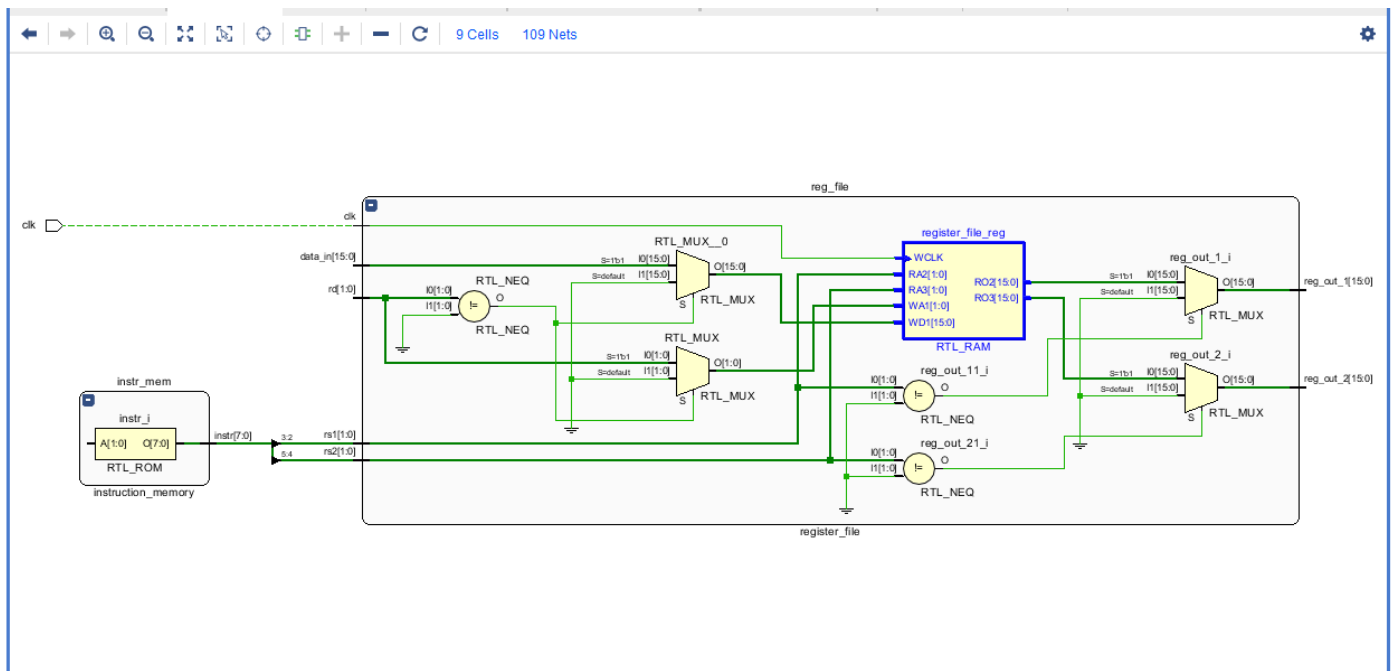
1df40	4d756464	61737369	725f416c	695f5369	Muddassir_Ali_Si
1df50	64646971	75690000	494e4600	00000000	ddiqui..INF.....
1df60	696e6600	00000000	4e414e00	00000000	inf.....NAN.....
1df70	6e616e00	00000000	30313233	34353637	nan.....01234567
1df80	38396162	63646566	00000000	00000000	89abcdef.....
1df90	30313233	34353637	38394142	43444546	0123456789ABCDEF
1dfa0	00000000	00000000	286e756c	6c290000	(null)..

Part 2:

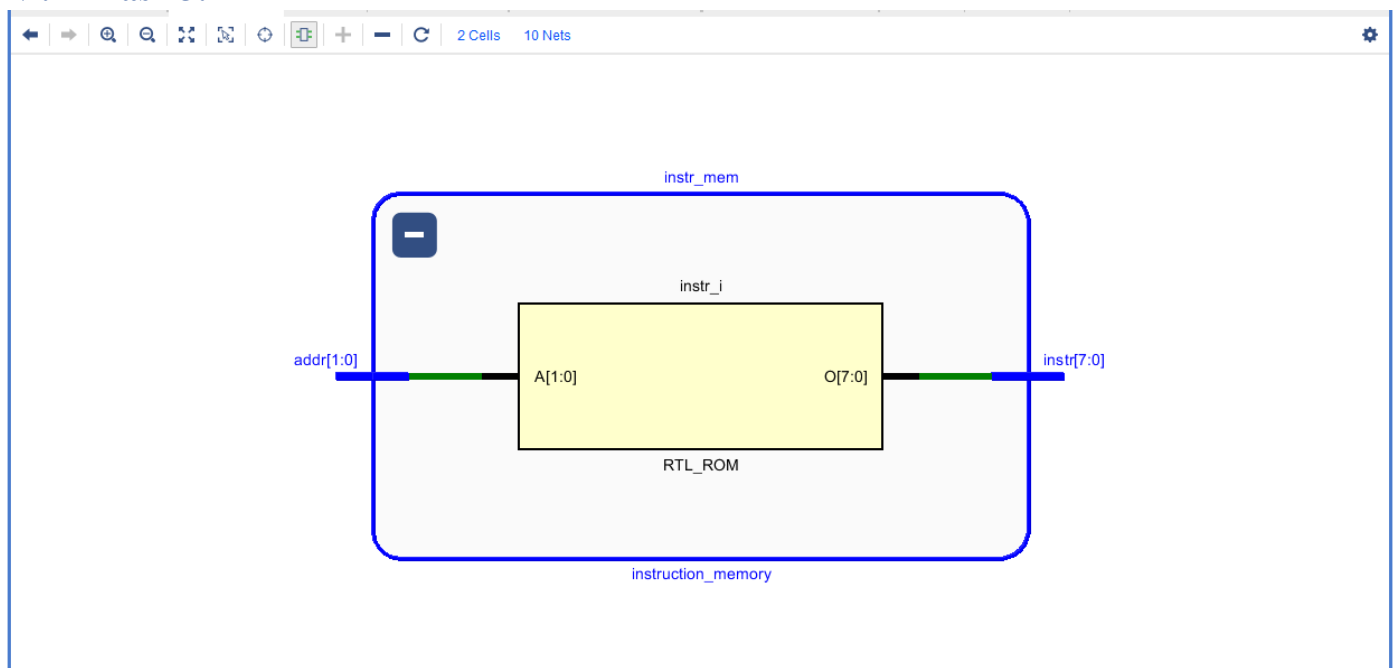
iv. Task 1:



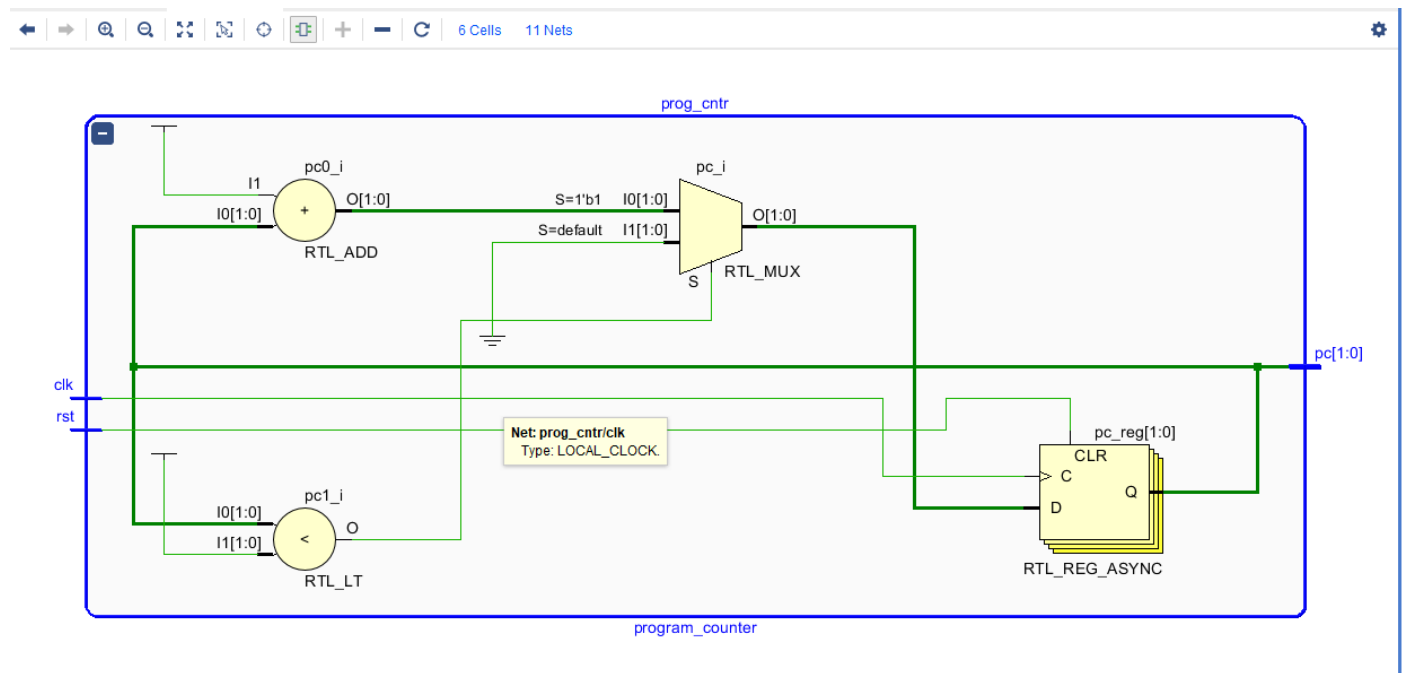
v. Task 2:



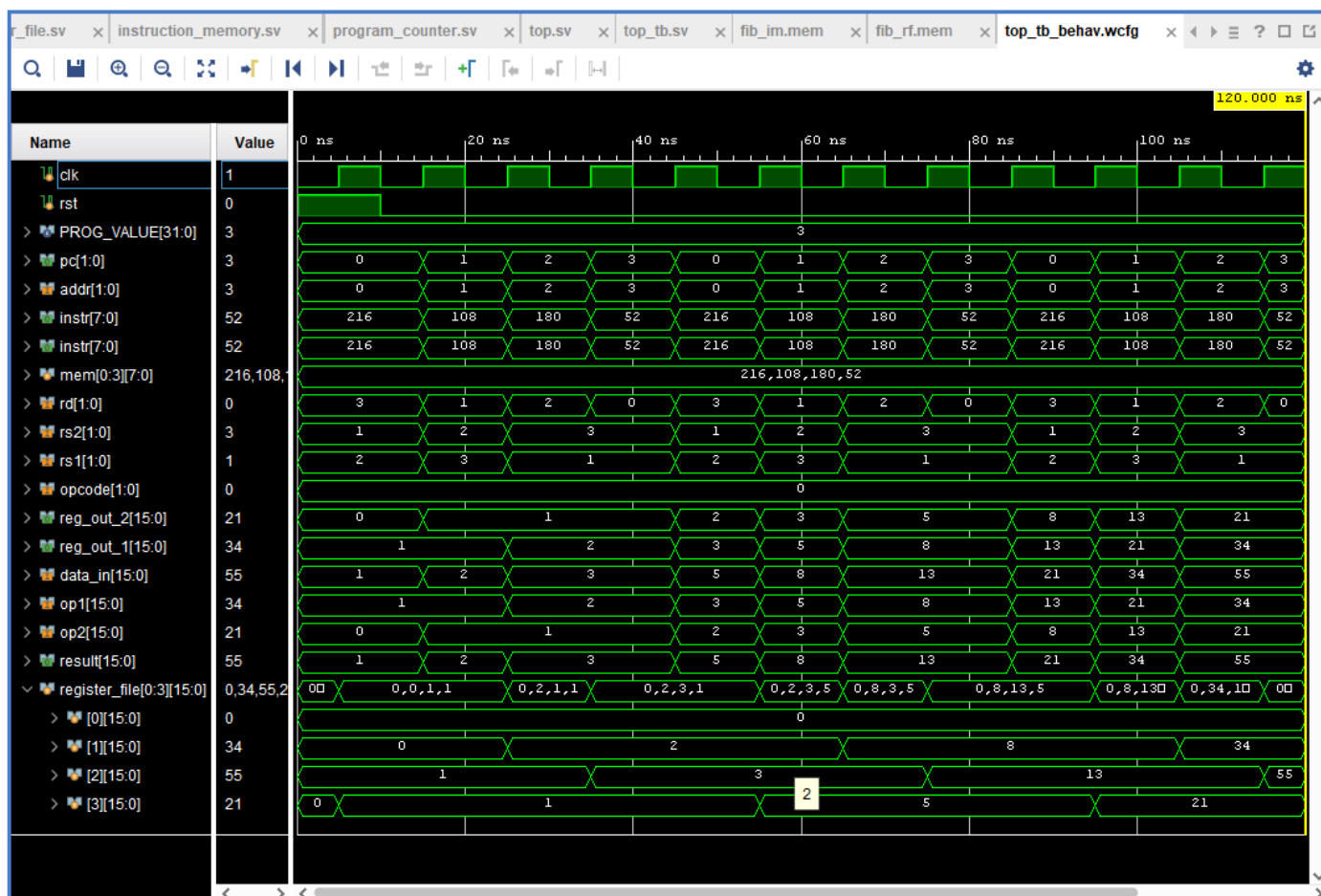
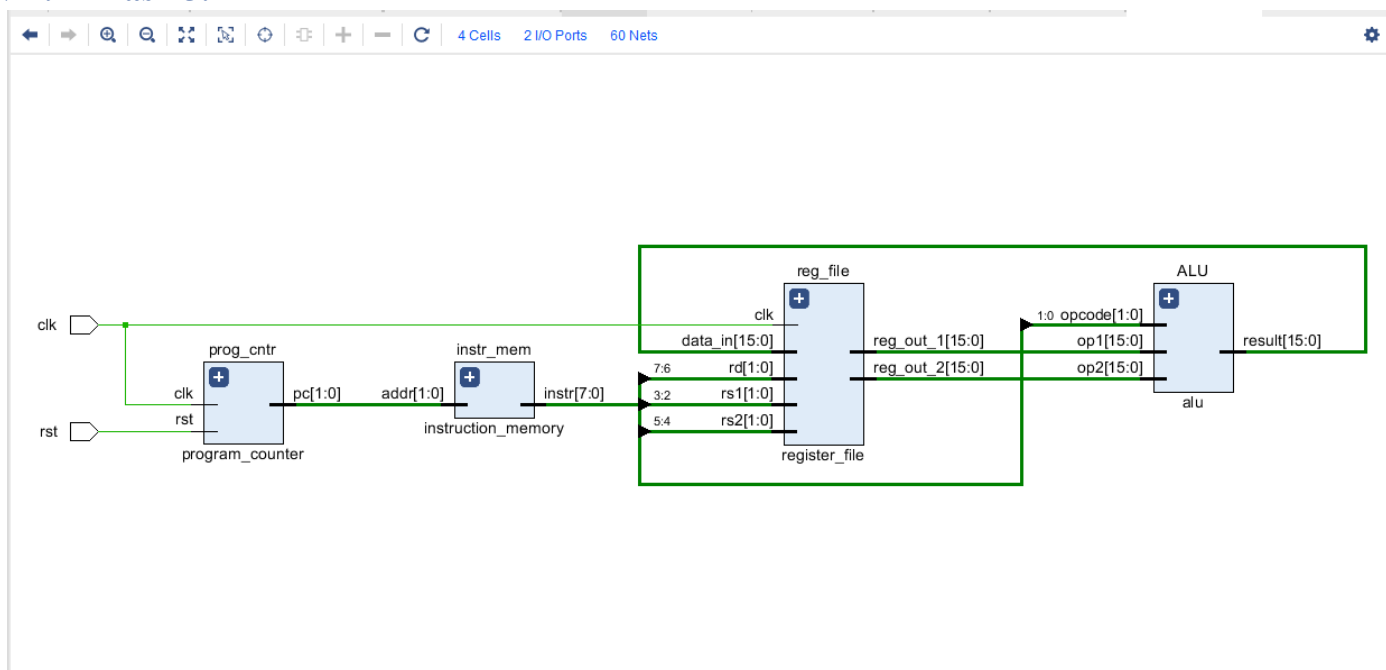
vi. Task 3:



vii. Task 4:



viii. Task 5:



2. Critical Analysis: (Write you critical analysis / conclusion here)

In this lab, we designed and implemented the fundamental modules of a simple processor, including a 16-bit ALU, a 16-bit Register File with four registers, an 8-bit Instruction Memory, and a Program Counter capable of addressing up to 4 instructions. All components were parameterized to allow flexibility in width and depth. After integration at the top level, the processor was initialized with fib_im.mem and fib_rf.mem, successfully executing the instructions. This lab provided practical insight into how parameterized hardware blocks interact to form a working microprocessor.